

C++ 프로그래밍

(예외 처리와 형 변환)

10주차

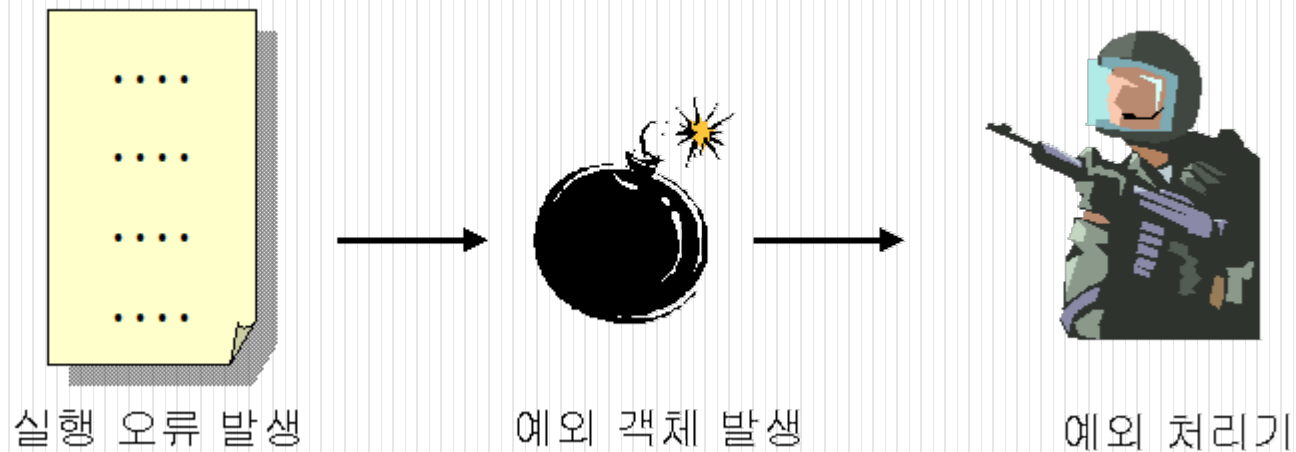
목차

- 예외 처리의 개념
- 예외 처리기 구조
- 예외 전달
- 다중 catch문장
- 자신의 예외 클래스 작성
- 형 변환
- 예외 처리 실습
 - 일반 코드
 - 고전적인 오류 처리
 - 함수를 사용한 오류 처리
 - try-catch블록 처리
 - 함수 내부에서 발생한(throw)한 예외 처리

예외 처리의 개념

- 예외란?
 - 잘못된 코드, 부정확한 데이터, 예외적인 상황에 의하여 발생하는 오류

예) 0으로 나누는 것과 같은 잘못된 연산이나 배열의 인덱스가 한계를 넘을 수도 있고, 디스크에서는 하드웨어 에러가 발생할 수 있다.



예외 처리기 구조

- 예외 처리기

```
try {
```



throw 예외;

```
}
```

```
catch {
```



```
}
```

try블록은 예외가 발생할 수 있는 블록입니다.

throw는 예외를 던진다.

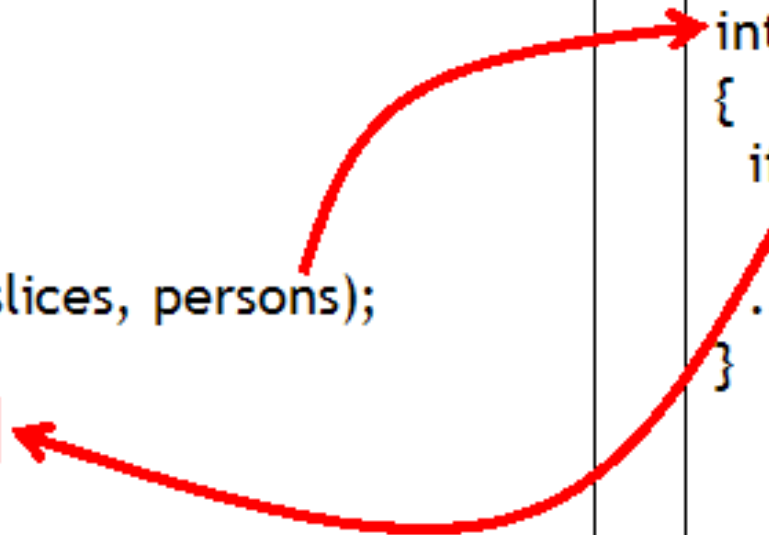
catch 블록은 try 블록에서 예외가 발생하면 처리합니다.

예외 전달

- 예외 전달

```
int main()
{
    try
    {
        dividePizza(slices, persons);
    }
    catch(int e)
    {
        cout << "예외 발생";
    }
}
```

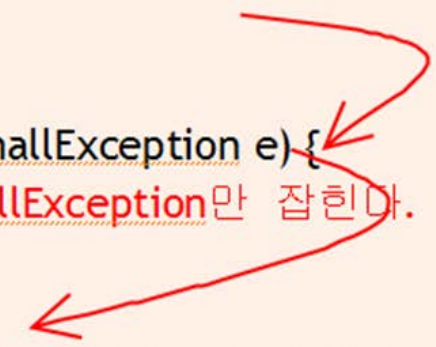
```
int dividePizza(int slices, int persons)
{
    if( persons == 0 )
        throw persons;
    ...
}
```



다중 catch문장

- 다중 catch문장
 - 하나의 try블록에서는 여러 개의 throw문장을 가질 수 있다.
 - 여러 가지 타입의 값을 처리하려면 여러 개의 catch블록을 두어야 한다.
 - 예를 들어서 피나 나누기 예제에서 사람 수가 0이 될 수도 있고 사람수가 음수가 될 수도 있다. 이것을 구분하여서 처리하려면 다음과 같이 두개의 catch블록을 정의하여야 한다.

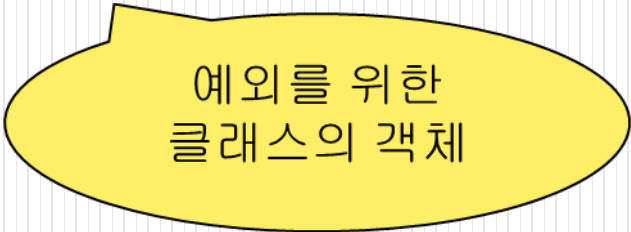
```
try {  
    getInput();  
}  
catch(TooSmallException e) {  
    //TooSmallException만 잡힌다.  
}  
catch(...) {  
    //TooSmallException을 제외한 나머지 예외들이 잡힌다.  
}
```



자신의 예외 클래스 작성

- 예외 클래스 작성
 - Throw문장은 클래스 타입의 객체도 던질 수 있다.
 - 예외 클래스도 단지 하나의 클래스에 불과

예) throw **NoPersonException**(persion);

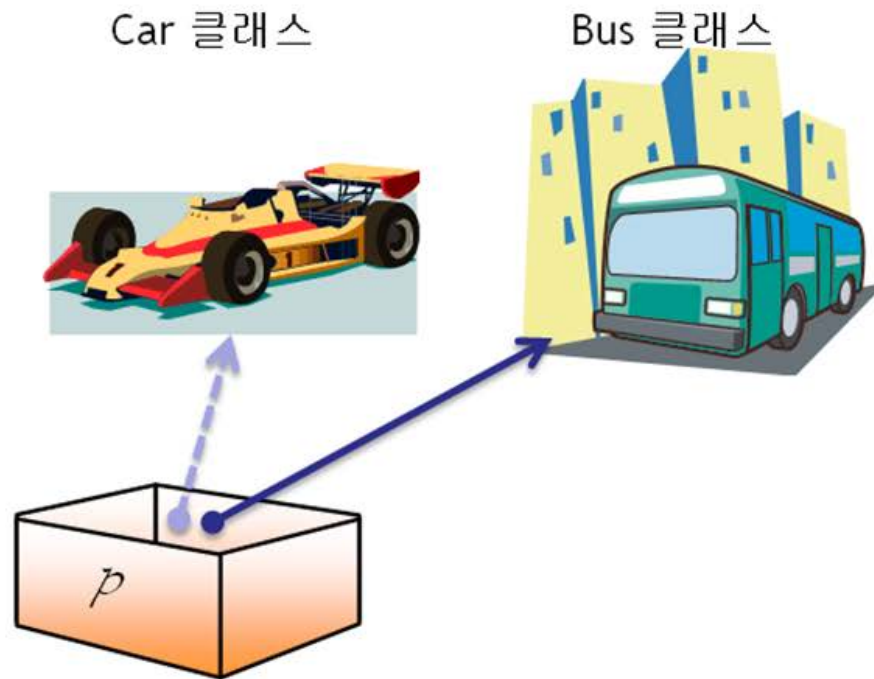


예외를 위한
클래스의 객체

형 변환

- 형 변환

```
double f = 3.141592;  
int i = (int) f;
```



형변환은 변수의
타입 (type)을 변
경하는 연산입니
다.



타입 정보

- 타입 정보
 - typeid연산자는 실행 시간에 객체의 타입을 식별할 수 있다.
 - typeid가 반환하는 값은 type_info에 대한 참조자 이다.

일반 코드

```
#include <iostream>
using namespace std;

int main ()
{
    int num1, num2, result;
    for (int i = 0; i < 5; i++)
    {
        cout << "Enter an integer: ";
        cin >> num1;
        cout << "Enter another integer: ";
        cin >> num2;
        result = num1 / num2; // The statement that may create exception
        cout << "The result of division is: " << result << endl;
    }
    return 0;
}
```

```
Enter an integer: 12
Enter another integer: 5
The result of division is: 2
Enter an integer: 10
Enter another integer: 3
The result of division is: 3
Enter an integer: 6
Enter another integer: 0
```

고전적인 오류 처리

```
#include <iostream>
using namespace std;

int main ( )
{
    int num1, num2, result;
    for (int i = 0; i < 4; i++)
    {
        cout << "Enter an integer: ";
        cin >> num1;
        cout << "Enter another integer: ";
        cin >> num2;
        if (num2 == 0)
        {
            cout << "Division cannot be done in this case." << endl;
        }
        else
        {
            result = num1 / num2;
            cout << "The result of division is: " << result << endl;
        }
    }
    return 0;
}
```

```
Enter an integer: 8
Enter another integer: 2
The result of division is: 4
Enter an integer: 7
Enter another integer: 1
The result of division is: 7
Enter an integer: 8
Enter another integer: 0
Division cannot be done in this case.
Enter an integer: 9
Enter another integer: 7
The result of division is: 1
```

함수를 사용한 오류 처리

```
#include <iostream>
using namespace std;

// Function declaration
int quotient (int first, int second);

int main ()
{
    int num1, num2, result;
    for (int i = 0; i < 3; i++)
    {
        cout << "Enter an integer: ";
        cin >> num1;
        cout << "Enter another integer: ";
        cin >> num2;
        result = quotient (num1, num2);
        if (result == -1)
        {
            cout << "Error, division by zero." << endl;
        }
        else
        {
            cout << "Result of division is: " << result << endl;
        }
    }
    return 0;
}
```

```
// Function definition
int quotient (int first, int second)
{
    if (second == 0)
    {
        return -1;
    }
    return (first / second);
}
```

```
Enter an integer: 6
Enter another integer: 5
The result of division is: 1
Enter an integer: 7
Enter another integer: 0
Error, division by zero.
Enter an integer: 8
Enter another integer: 2
The result of division is: 4
```

try-catch블록 처리

```
#include <iostream>
using namespace std;

int main ()
{
    int num1, num2, result;
    for (int i = 0; i < 3; i++)
    {
        cout << "Enter an integer: ";
        cin >> num1;
        cout << "Enter another integer: ";
        cin >> num2;
        // The try-catch block
        try
        {
            if (num2 == 0)
            {
                throw 0; // An object of type integer is thrown
            }
            result = num1 / num2;
            cout << "The result is: " << result << endl;
        }
        catch (int x)
        {
            cout << "Division by zero cannot be performed." << endl;
        }
    }
    return 0;
}
```

```
Enter an integer: 6
Enter another integer: 5
The result is: 1
Enter an integer: 7
Enter another integer: 0
Division by zero cannot be performed.
Enter an integer: 8
Enter another integer: 2
The result is: 4
```

함수 내부에서 발생한(throw)한 예외 처리

```
#include <iostream>
using namespace std;

int quotient (int first, int second); // Function declaration

int main ()
{
    int num1, num2, result;
    for (int i = 0; i < 3; i++)
    {
        cout << "Enter an integer: ";
        cin >> num1;
        cout << "Enter another integer: ";
        cin >> num2;
        // The try-catch block
        try
        {
            cout << "Result: " << quotient (num1, num2) << endl;
        }
        catch (int ex)
        {
            cout << "Division by zero cannot be performed." << endl;
        }
    }
    return 0;
}
```

```
// Function definition
int quotient (int first, int second)
{
    if (second == 0)
    {
        throw 0;
    }
    return first / second;
}
```

```
Enter an integer: 12
Enter another integer: 4
Result: 3
Enter an integer: 16
Enter another integer: 0
Division by zero cannot be performed.
Enter an integer: 7
Enter another integer: 2
Result: 3
```